2. Análisis de las interfaces

Interfaz	Para qué sirve	Métodos principales
IGraph<V, D>	Operaciones comunes a cualquier grafo	agregarVertice, agregarArista, adyacencias, existeArista, esConexo, tieneCiclos
IDirectedIGraph<V, D>	Agrega lo propio de un grafo dirigido	successors, predecessors, gradoDeSalida, gradoDeEntrada
IDirectedGraphAlgorithms	Algoritmos sobre grafos dirigidos	dijkstra, floyd, warshall, centro, excentricidad, todos los caminos, DFS, BFS, topológica

IGraph

Es la interfaz base. V es el tipo de los vértices y D el dato que guarda cada arista. Lo que más nos llamó la atención fue lo siguiente:

- Para agregar un vértice o una arista se pasa el vértice (V), pero para buscar, eliminar o consultar se pasa un Comparable<V>, que funciona como un criterio de búsqueda. Para encontrar el vértice que cumple el criterio hay que ir comparando vértice por vértice, así que esas operaciones cuestan al menos O(V).
- Las aristas dirigidas (DirectedEdge) se comparan solo por origen y destino, sin mirar el dato. Por eso no puede haber dos aristas entre los mismos vértices, y agregarArista devuelve false si ya existía.
- Algunos métodos ya vienen implementados con default (por ejemplo existeVertice y cantidadDeAristas). Funcionan, pero pueden ser lentos, y conviene sobreescribirlos en nuestra implementación.
- En un grafo dirigido, esConexo() puede entenderse de dos maneras (ignorando el sentido de las aristas o respetándolo). Hay que elegir una y documentarla.

IDirectedIGraph

Extiende a IGraph y agrega successors (los vértices a los que se llega desde uno) y predecessors (los vértices que llegan a uno). gradoDeSalida y gradoDeEntrada vienen calculados como el tamaño de esos conjuntos. Lo importante es que **predecessors es difícil de resolver si solo se guardan las aristas salientes**, porque habría que revisar todo el grafo. Esto influye en la elección de la estructura (sección 3).

IDirectedGraphAlgorithms

- Los algoritmos no están dentro del grafo: reciben el grafo por parámetro. Los resultados se devuelven con interfaces propias (IDijkstraResult, IFloydWarshallResult) y con la clase Path.
- Los algoritmos que usan pesos (Dijkstra, Floyd, Warshall, centro, excentricidad y todos los caminos) exigen que el dato de la arista sea un WeightedEdge. DFS, BFS y la clasificación topológica no usan pesos.

- DFS y BFS tienen una versión con vértice de origen y otra sin origen. La segunda llama a la primera para cada vértice, y como no recuerda lo ya visitado, puede pasar el mismo vértice al Consumer varias veces. Habría que revisar si eso es lo que se busca.
- La interfaz no aclara qué pasa en algunos casos: un vértice al que no se puede llegar en Dijkstra, o un grafo con ciclos en la clasificación topológica. Esas decisiones las tenemos que tomar nosotros y probarlas con tests.

3. Colecciones de Java para implementar IDirectedIGraph

Alternativas

Alternativa	Cómo se guarda	Colecciones de Java
Lista de adyacencia	Cada vértice tiene la lista de sus aristas salientes	Map<V, List<Edge>> con HashMap o LinkedHashMap, y ArrayList o LinkedList
Mapa de mapas	Cada vértice tiene un mapa destino → arista	Map<V, Map<V, Edge>>
Matriz de adyacencia	Una matriz V×V con la arista (o null)	Edge[][] y un Map<V, Integer> para saber la posición de cada vértice

Orden de tiempo

Operación	Lista de adyacencia	Mapa de mapas	Matriz
Agregar vértice	O(1)	O(1)	O(V) (hay que agrandar la matriz)
Agregar arista	O(grado) (hay que ver que no exista)	O(1)	O(1)
¿Existe la arista?	O(grado)	O(1)	O(1)
Sucesores / adyacentes	O(grado)	O(grado)	O(V)
Predecesores	O(V+E)	O(V)	O(V)
Recorrido (DFS / BFS)	O(V+E)	O(V+E)	$O(V^2)$

Consumo de memoria

- **Lista de adyacencia y mapa de mapas:** la memoria es proporcional a V + E, es decir, solo se guardan las aristas que existen.
- **Matriz:** la memoria es proporcional a V^2, aunque haya pocas aristas. Con V = 10.000 serían 100 millones de casilleros, y si cada referencia ocupa unos 4 bytes serían aproximadamente 400 MB solo para la matriz. Para grafos con pocas aristas no conviene.

- **Mapa de mapas contra listas:** el mapa de mapas permite consultar una arista en O(1), pero cada HashMap crea objetos internos por cada entrada, así que gasta bastante más memoria que una lista.
- **ArrayList contra LinkedList:** la LinkedList crea un nodo por cada elemento (con referencias extra al anterior y al siguiente), mientras que la ArrayList guarda las referencias juntas en un arreglo. Para recorrer adyacencias la ArrayList usa menos memoria y es más rápida.
- **HashMap contra LinkedHashMap:** el segundo recuerda el orden de inserción, a cambio de un poco más de memoria. Es útil para que los tests de DFS y BFS den siempre el mismo orden.

Limitaciones

El análisis es teórico: no medimos tiempos ni memoria reales sobre nuestra implementación. En el Desafío 3 podemos medir y comparar. Una mejora que queremos evaluar ahí es evitar el recorrido de vértices que hoy exige buscar por Comparable, porque ese costo se repite en cada operación.

Conclusión

Las interfaces separan bien las operaciones del grafo de los algoritmos, pero dejan varias decisiones abiertas (conectividad en dirigidos, casos sin camino, ciclos en la topológica). Para la estructura de datos, la lista de adyacencia con un índice inverso es un buen equilibrio entre memoria y tiempo para grafos dispersos, que es lo que esperamos en este proyecto.